Objetivo

Receberei uma imagem já segmentada, depois meu modelo vai separar o rosto em landmarks(pontos em áreas importantes) nos olhos, boca e nariz. Passando essas informações para o modelo em si reconhecer.

Pipeline:
PEGA ROSTO RECORTADO
RECONHECIMENTO FACIAL (ArcFace)
Comparar distâncias euclidianas e checar se bate

ArcFace

Artigo do ArcFace que eu extrai tudo que vou dizer aqui
No geral ArcFace resolve problemas de classificação e generalização, no sentido de separa os embeddings de cada pessoa perto das mesmas pessoas no espaço vetorial e longe se elas forem pessoas diferentes.

Eles usam o termo backbone para falar sobre uma área que eu fiz assim no MNIST

```python
class ModeloMNIST(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.bloco1 = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=(3, 3), bias=True),
            nn.MaxPool2d(kernel_size=(2, 2)), # Aqui eu to usando MaxPool que é pegar o
            nn.Sigmoid(),
            nn.BatchNorm2d(num_features=32),
        )
        self.bloco2 = nn.Sequential(
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=(3, 3), bias=True),
            nn.MaxPool2d(kernel_size=(2, 2)), # Aqui eu to usando MaxPool que é pegar o
            nn.Sigmoid(),
            nn.BatchNorm2d(num_features=64),
        )
        self.mlp = nn.Sequential(
            nn.Linear(in_features=1600, out_features=32),
            nn.Sigmoid(),
            nn.Linear(in_features=32, out_features=10),
        )
    def forward(self, dados: torch.Tensor) -> torch.Tensor:
        saida = self.bloco1(dados)
        saida = self.bloco2(saida)
        saida = self.mlp(saida.flatten(1))
        return saida
```

A tradução liteal é espinha dorsal, eu posso usar como Rede base.

Eles propoem uma loss function chamada de "ArcFace" 😄
Ele usa o "Additive Angular Margin Loss" para reduzir as distâncias inta e inter classes já mencionada anteriormente.

Funções que eles eles passam no quesito sucesso é a SoftMax Loss que é descrita no mesmo artigo como

$$L_1 = -\log \frac{e^{W_{y_i}^T x_i + b_{y_i}}}{\sum_{j=1}^{N} e^{W_j^T x_i + b_j}}$$

Olhando essa função é possível perceber que ela é feita a partir da SoftMax function e da Entropia Cruzada. Ela é boa em fazer o reconhecimento porém na "deep face recognition" ela perfoma mal em classifica variações de aparência como idade e poses diferentes pois as classes são formadas pertos uma das outras pois ele não força a separação dessas classes. Ela funciona a partir da normalização em um vetor de probabilidades e depois do cálculo atráves do que é e do esperado.

O artigo cita o Triplet Loss também. Nele você coloca três imagens, uma âncora ou numa tradução melhor, pivô. Uma dessa mesma pessoa e outra de aluém diferente dessa imagem. A função funciona e separa bem as pessoas resolvendo o problema da SoftMax Loss, porém ele tem um custo elevado exponencial em datasets grandes, o que acaba não sendo viável em bancos de dados como LFW.

$$\text{Loss} = \sum_{i=1}^{N} \left[\| f_i^a - f_i^p \|_2^2 - \| f_i^a - f_i^n \|_2^2 + \alpha \right]_+$$

Sendo "a" a âncora, "p" a positiva e "n" a negativa.

O artigo cita ainda a SphereFace Loss e a CosFace Loss e foram justamente dessas funções que ele se inspirou pois ele resgata a "angular margin" e a "arc-cosine" usadas nelas porém sem uma notável instabilidade da SphereFace por exemplo..

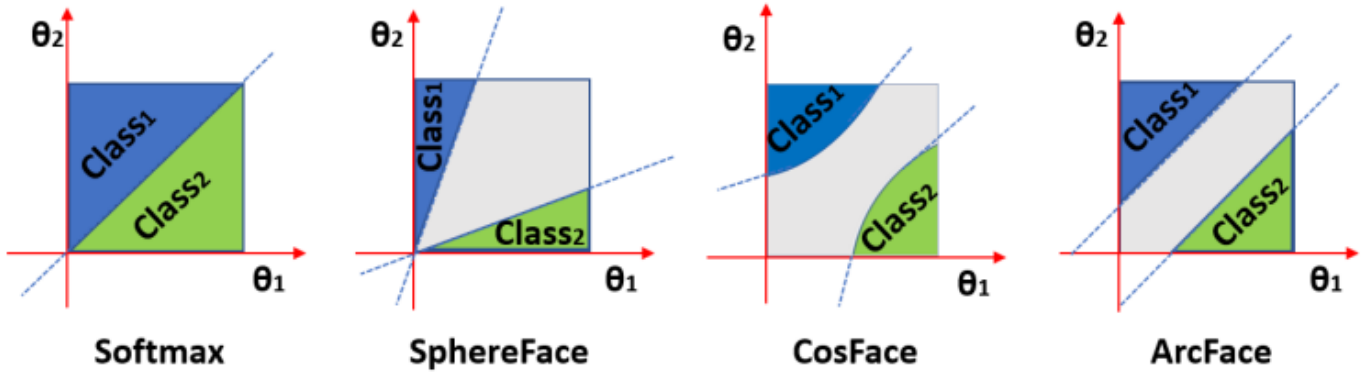
$$L = -\log \left(\frac{e^{s \cdot \cos(\theta_{y_i} + m)}}{e^{s \cdot \cos(\theta_{y_i} + m)} + \sum_{j \neq y_i} e^{s \cdot \cos(\theta_j)}} \right)$$

Como uma ela é modificação direta da SoftMax Loss, ela também mede o erro(cross-entropy) e transforma os resultados em probabilidades(SoftMax), porém agora incorporando elementos da SphereFace e da CosFace, dessa maneira trabalhando também com ângulos como a SphereFace trabalha com uma hiperesfera.

Separando o que ocorre numa linha do tempo, ele gera de inicio uma normalização dos vetores e o resultado é passado com ângulos. A partir desse resultado são feitos os scores e depois uma margem, forçando uma separação ainda maior entre a classe certa e da classe errada. Logo após ele passa por uma mudança de escala(pesquisar melhor essa mudança de escala, "s" da fórmula), passando pelo softmax no final para achar uma probabilidade e assim ao final obtendo o loss pelo "-L = -log(p_y)" restante.

Na fórmula o $\cos(\theta)$ mede a similaridade (o nome dessa técnica de separação é cosine similarity) , o "m" força uma separação maior, o "s" melhora o treino com escala, o "$e^{s \cdots}$" transforma em probabilidade e o "log" penaliza o erro.

Fronteira de decisão : "Decision margins of different loss functions under binary classification case"



MediaPipe

Modelo recente(2019) OpenSource da Google que detecta pontos cruciais de um rosto.
ArcFace já faz isso então não tem uso para minha pipeline. Se for útil eu consigo usar depois

Perguntas para a próxima reunião:
Ele vai reconhecer que é um rosto e depois vai ver um banco de dados para achar quem é a pessoa?

Qual banco de dados nós testamos os modelos? VGG Face, LFW...?

videos e livros importantes

Face recognition
Python Tutorial
Applied Machine Learning and AI for Engineers - Jeff Prosise